

Lecture 8: Loop Invariants

September 17, 2026

Lecturer and scribe: Karthik Gajulapalli

We prove two algorithms correct using **loop invariants**: statements about the processed prefix that we establish by induction and use when the loop ends.

1 Maximum Contiguous Sum

Suppose we are given a nonempty array of integers

$$a_1, a_2, \dots, a_n.$$

The entries may be positive, zero, or negative. The **maximum contiguous sum problem**, also called the **maximum subarray problem**, asks us to choose indices $1 \leq \ell \leq r \leq n$ for which

$$a_\ell + a_{\ell+1} + \dots + a_r$$

is as large as possible. We want the maximum sum and a block that attains it. **Contiguous** means consecutive: we cannot skip entries inside the chosen block.

For example, consider

$$[4, -6, \underbrace{3, 2, -1, 4}_{\text{sum}=8}, -8].$$

The maximum contiguous sum is 8, achieved by the block from position 3 through position 6. Choosing only positive entries would give $4 + 3 + 2 + 4 = 13$, but those entries do not form a contiguous block.

We require the block to be **nonempty**. Thus for the array $[-5, -2, -7]$, the answer is -2 , rather than 0.

There are n blocks starting at position 1, $n - 1$ starting at position 2, and so on. By the sum-of-integers formula from Lecture 6, the total number of candidate blocks is

$$n + (n - 1) + \dots + 1 = \frac{n(n + 1)}{2}.$$

Can we find the best block without checking each candidate separately? What information should we keep after reading the first i entries, and how should we update it when we read entry $i + 1$?

We will develop an algorithm and analyze its correctness and running time. Induction will let us prove that the information we keep after each step is correct, connecting our earlier proofs to reasoning about a program.

1.1 The Naive Approach

Try all $n(n+1)/2$ intervals and keep the largest sum. Summing each interval from scratch takes $O(n^3)$ time in total. We can improve this to $O(n^2)$ time by fixing a starting position and extending the endpoint, updating a running sum. This uses $O(1)$ extra space.

1.2 Kadane's Algorithm

A negative partial sum cannot improve a future extension, so we discard it and start fresh. We use positions 1 through n , with $A[i] = a_i$.

```
Kadane(A):                                # assume len(A) >= 1
    max_sum = -infinity
    cur_sum = 0
    for i = 1, 2, ..., len(A):
        cur_sum = cur_sum + A[i]
        max_sum = max(cur_sum, max_sum)
        cur_sum = max(cur_sum, 0)
    return max_sum
```

1.3 Tracing the Example

Consider the input

[−2, 1, −3, 4, −1, 2, 1, −5, 4].

The middle column records `cur_sum` immediately after adding $A[i]$; the final column records it after the reset. Initially, `max_sum` = $-\infty$ and `cur_sum` = 0.

i	$A[i]$	After addition	<code>max_sum</code>	After reset
1	−2	−2	−2	0
2	1	1	1	1
3	−3	−2	1	0
4	4	4	4	4
5	−1	3	4	3
6	2	5	5	5
7	1	6	6	6
8	−5	1	6	1
9	4	5	6	5

The negative sum at position 3 is discarded, so position 4 starts a new interval. At position 7 we reach $4 - 1 + 2 + 1 = 6$. Notice that a *negative entry* does not by itself trigger a reset: the −1 at position 5 leaves a positive running sum of 3. The maximum is 6, attained by the block [4, −1, 2, 1] in positions 4 through 7.

2 Proving Kadane's Algorithm Correct by Induction

The key observation is simple: a block ending at position $i + 1$ either starts there or extends a block ending at position i .

Claim 1 (Kadane invariant). *On every iteration i , for $1 \leq i \leq n$:*

1. *Immediately after adding $A[i]$, **cur_sum** is the largest sum of a nonempty block ending at position i .*
2. *After updating **max_sum**, it is the largest sum of a nonempty block anywhere in the first i entries.*

Proof by induction on i . **Base case** ($i = 1$). The only nonempty block is $[a_1]$. The addition sets **cur_sum** to a_1 , and the next line sets **max_sum** to a_1 . Both statements hold.

Inductive step. Assume both statements hold on iteration i , where $i < n$. Let S be the best sum ending at i . By the induction hypothesis and the reset, the next iteration starts with **cur_sum** = $\max(S, 0)$.

A block ending at $i + 1$ has just two possibilities:

- **Start fresh:** take $[a_{i+1}]$, with sum a_{i+1} .
- **Extend:** append a_{i+1} to a block ending at i . The best such sum is $S + a_{i+1}$.

Thus the best sum ending at $i + 1$ is

$$\max(a_{i+1}, S + a_{i+1}) = \max(0, S) + a_{i+1},$$

exactly what the next addition computes. This proves the first statement.

The best block in the first $i + 1$ entries either lies entirely in the first i entries or ends at $i + 1$. Taking the maximum of the old **max_sum** and the new **cur_sum** therefore proves the second statement. Induction completes the proof. $\square$

At $i = n$, the second statement says that the returned value is the maximum nonempty contiguous sum of the entire array.

Recovering a block. To return the block as well, keep its current starting position, initially 1. Whenever a negative running sum is reset, move that starting position to $i + 1$. Whenever the running sum improves **max_sum**, save the current start and the endpoint i . The saved endpoints describe a block attaining the returned sum.

Cost. Each iteration does a constant amount of work, so the algorithm uses $O(n)$ time and $O(1)$ extra space, counting arithmetic and comparisons as constant-time operations.

3 Finding a Majority Element

An element m is a **majority element** of an array of length n if it occurs *strictly more than* $n/2$ times. There can be at most one majority: two distinct elements each occurring more than $n/2$ times would together occupy more than n positions.

Input	Frequencies	Majority
$[a, b, a, c, a, b, a]$	$a : 4, b : 2, c : 1$	a
$[a, a, b, b]$	$a : 2, b : 2$	None
$[a, a, b, b, c]$	$a : 2, b : 2, c : 1$	None

Being tied for most frequent is not enough; even a unique most frequent element need not be a majority. For example, a occurs three times in $[a, a, a, b, b, c, c]$, but $3 \not> 7/2$.

3.1 A Simple Counting Algorithm

Keep a dictionary mapping each distinct value to its frequency.

```
MajorityByCounting(A):
    frequency = empty dictionary, with default value 0
    for x in A:
        frequency[x] = frequency[x] + 1
    for x in frequency:
        if frequency[x] > len(A) / 2:
            return x
    return NONE
```

There are at most n dictionary entries. With constant-time dictionary operations, this takes $O(n)$ time and $O(n)$ extra space. A standard hash table gives the time bound in expectation. We now seek the same linear running time using only a candidate and a counter.

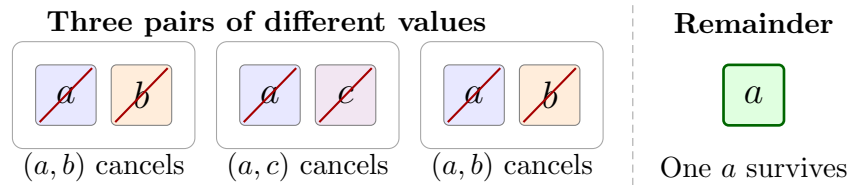
3.2 Cancellation and the Partition Lemma

Pair two *different* values and remove both occurrences. If a majority m exists, this cannot destroy it. If m appears $f > n/2$ times, removing an unequal pair removes at most one copy of m . Its new frequency is therefore at least $f - 1$, and

$$f - 1 > \frac{n}{2} - 1 = \frac{n - 2}{2}.$$

So m is still a strict majority of what remains.

For example, we can cancel two (a, b) pairs and one (a, c) pair from $[a, b, a, c, a, b, a]$, leaving one a . The pairs need not have been adjacent: we are counting occurrences, not preserving intervals.



A **cancellation partition** divides the occurrences in an array into two parts:

- a part P that is further divided into disjoint pairs, each containing two different values;
- a remainder R consisting entirely of copies of a single value c , or an empty remainder.

Every occurrence belongs to exactly one pair or to the remainder. Repeated values are separate occurrences, so they may belong to different pairs.

Lemma 1 (The surviving value is forced). *If an array has a majority element m , a cancellation partition exists. In every such partition, the remainder is nonempty and consists only of copies of m .*

Proof. Existence. Let m occur $f > n/2$ times. There are $n - f$ occurrences different from m . Pair each of them with a distinct copy of m . Since $f > n - f$, there are enough copies, and this leaves $f - (n - f) = 2f - n > 0$ copies of m in the remainder.

The surviving value. Consider *any* cancellation partition, with p pairs and r unpaired occurrences. Then $n = 2p + r$. Each unequal pair contains at most one copy of m . If the remainder were empty, or if its common value were different from m , all copies of m would be in the pairs. There would be at most

$$p = \frac{n - r}{2} \leq \frac{n}{2}$$

copies of m , contradicting the majority assumption. Thus the remainder is nonempty and its common value is m . $\square$

What is unique? The surviving *value* is unique when a majority exists. The partition itself need not be unique, and even the number of survivors can differ. For $[a, a, a, b, c]$, both of the following are valid:

Unequal pairs	Remainder
$(a, b), (a, c)$	$[a]$
(b, c)	$[a, a, a]$

Both leave only a , the original majority. Thus the lemma guarantees one possible surviving value, not one possible partition.

3.3 Boyer–Moore Majority Vote

Maintain a **candidate** and a nonnegative **count**. Think of **count** as the number of uncanceled copies of the candidate. An equal value adds a copy; a different value cancels one copy. If no copies remain, the next value becomes the candidate.

```
MajorityCandidate(A):
    candidate = NONE
    count = 0
    for x in A:
        if count == 0:
            candidate = x
            count = 1
        else if x == candidate:
            count = count + 1
        else:
            count = count - 1
    return candidate
```

If a majority is promised to exist, this returned candidate is the answer. We use this assumption in the correctness proof below. The symbol **NONE** means that no candidate has yet been chosen.

Example with a Majority

On $[a, b, a, c, a, b, a]$, the state after each step is as follows. The displayed pairs and remainder explain the algorithm; they are not stored by it.

Read	Candidate	Count	Pairs so far	Remainder
a	a	1	None	$[a]$
b	a	0	(a, b)	$[\]$
a	a	1	(a, b)	$[a]$
c	a	0	$(a, b), (a, c)$	$[\]$
a	a	1	$(a, b), (a, c)$	$[a]$
b	a	0	$(a, b), (a, c), (a, b)$	$[\]$
a	a	1	$(a, b), (a, c), (a, b)$	$[a]$

The final candidate is a . Its counter is 1, although a occurs 4 times: **count** measures uncanceled copies, not frequency. Whenever the count is zero, the stored candidate has no significance.

A Candidate Change and an Input with No Majority

For $[b, a, a]$, the states (`candidate`, `count`) are

$$(b, 1) \longrightarrow (b, 0) \longrightarrow (a, 1).$$

The first two entries cancel, and the last entry chooses the new candidate a , which is the majority.

For $[a, a, b, b, c]$, the states are

$$(a, 1) \longrightarrow (a, 2) \longrightarrow (a, 1) \longrightarrow (a, 0) \longrightarrow (c, 1).$$

The remainder is $[c]$, but c occurs only once in the original array. A nonempty remainder alone does *not* establish a majority. The cancellation lemma assumes that a majority already exists.

3.4 The Invariant and the Correctness Proof

Claim 2 (Boyer–Moore invariant). *After processing exactly k entries, for every $0 \leq k \leq n$, the processed prefix has a cancellation partition into:*

1. *disjoint pairs of different values; and*
2. *exactly `count` unpaired copies of `candidate`.*

If `count` = 0, the remainder is empty and the candidate may be ignored.

Proof by induction on k . **Base case** ($k = 0$). The processed prefix is empty. Take no pairs and an empty remainder, matching `count` = 0.

Inductive step. Assume the invariant after k entries, and let $x = A[k + 1]$ be the next value. Fix a partition supplied by the induction hypothesis. There are three cases, matching the code.

1. **The old count is zero.** The old remainder is empty. Keep every old pair and put x alone in the new remainder. Setting `candidate` = x and `count` = 1 records exactly this partition.
2. **The old count is positive and x equals the candidate.** Add x to the remainder. Its elements are still all equal to the candidate, and its size increases by one, exactly as the counter does. The pairs are unchanged.
3. **The old count is positive and x differs from the candidate.** Remove one occurrence from the old remainder and pair it with x . Such an occurrence exists because the old count is positive, and the new pair has different values. The remaining unpaired occurrences still all equal the candidate; their number decreases by one, exactly as the counter does. If it reaches zero, the new remainder is empty.

In every case, each processed occurrence belongs to exactly one pair or to the remainder. Thus the invariant holds after $k + 1$ entries. Induction proves the claim. $\square$

Claim 3 (Correctness when a majority exists). *If the input array has majority element m , the algorithm returns m .*

Proof. At termination, Claim 2 supplies a cancellation partition of the *entire* array. By Lemma 1, its remainder is nonempty and consists only of m . By the invariant, that same remainder consists of `count` copies of `candidate`. Therefore `count` > 0 and `candidate` $= m$, which is returned. $\square$

The invariant is valid even for prefixes with no majority. It does not say that the current candidate is a majority of the processed prefix. We are able to guarantee, that we find the majority when it exists because of the cancellation lemma.

Cost. The algorithm scans the array once in $O(n)$ time and stores only a candidate and a counter, so it uses $O(1)$ extra space.